

# Synthesizable SystemVerilog — 2-Page Quiz Cheat Sheet

Paper + presentation compressed into a cram sheet. Focus: what is synthesizable, why it helps, and the gotchas professors love.

**Core thesis:** SystemVerilog is not just for verification. It extended Verilog for both design and verification, and many of those design-side extensions are synthesizable. The biggest benefits are cleaner intent, fewer mismatches, fewer lines of RTL, and more compile-time error checking.

| High-yield construct   | Why it matters                                                               | Quiz caveat / memory hook                                                                              |
|------------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>logic</b>           | Default 4-state type for most ports/signals; clearer than wire/reg.          | Context infers net vs variable. Authors recommend logic almost everywhere; avoid 2-state RTL types.    |
| <b>enum</b>            | Restricts a variable to legal named values; ideal for FSM states.            | Label size must match base type. Huge win because many raw-parameter FSM bugs become syntax errors.    |
| <b>struct</b>          | Bundles related signals under one name.                                      | Can assign/copy/pass the whole bundle. Use packed struct only when contiguous bit layout matters.      |
| <b>packed union</b>    | One storage space viewed in multiple packed formats.                         | Only packed unions are synthesizable; all members must be same size and packed.                        |
| <b>typedef</b>         | Defines reusable bus/opcode/packet types once.                               | Prevents width/type mismatches; paper explicitly says use typedef liberally.                           |
| <b>package</b>         | Project-wide home for shared types, params, helper functions.                | Better than <code>`include</code> because scope is controlled. Must import symbols.                    |
| <b>packed arrays</b>   | Vectors can be subdivided, e.g. <code>logic [3:0][7:0]</code> .              | Great for byte/field selects inside one vector; packed dimensions go before the name.                  |
| <b>unpacked arrays</b> | Arrays of bytes/structs/types can be copied and assigned as a whole.         | Whole-array assign/copy/list assignment is synthesizable; <code>foreach</code> was not supported here. |
| <b>relaxed ports</b>   | Arrays, slices, structs, unions, and typedef'd types can pass through ports. | Synthesis may flatten/rename generated ports, so netlists may not resemble source RTL.                 |
| <b>interfaces</b>      | Group related bus signals into one port.                                     | Reduces repetitive wiring and inter-module mismatches; use interface/modport types intentionally.      |

## 2-state vs 4-state

- **logic is 4-state**; bit/byte/shortint/int/longint are 2-state in simulation.
- **Synthesis treats 2-state types like storage**, so sim can hide X/power-up issues.
- **Best practice**: logic for almost everything; int mostly for loop iterators.

## Arrays + whole-object moves

- **Major practical gain**: whole-array and whole-struct copy/list assignment reduces boilerplate.
- **Examples**: `packet <= '{default:0}; a2 = a1; q_out <= transform(d_in);`
- **Non-synth extras then**: `foreach`, array sorting/locator helpers, bit-stream casting.

## Syntax to recognize fast

- **logic [3:0][7:0] a;** 32-bit vector split into four bytes.
- **a1 = '{7,3,0,5},{'default:'1};** assign full array in one shot.
- **import pkg::\*;** pull package names into scope.

## Myth buster

- **Since 2009**, the IEEE name is SystemVerilog, not “current Verilog.”
- **So the claim** “Verilog synthesizes but SystemVerilog does not” is the myth.
- **Quiz angle**: they may ask this directly.

## Paper-only array/port details

- C-style array declarations are synthesizable.
- Array copies and array value lists are synthesizable.
- Arrays can be passed through ports and functions if dimensions/types match.
- Array query helpers like `$low/$high/$size` help avoid hard-coded bounds.

## Enum / struct / union gotchas

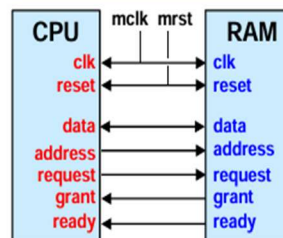
- enum defaults to int unless you give an explicit base type.
- enum labels must be unique and assignment rules are strict.
- Structures can be assigned as a whole; packed layout matters when using unions.
- DC had a limitation with assigning value lists to structs containing union members.

## Port / netlist memory hooks

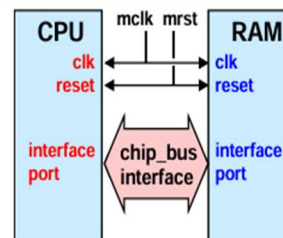
- Dot-name and dot-star reduce repetitive instance wiring.
- They also enforce explicit declarations and size matching.
- Typed/array/struct ports may come out flattened or renamed after synthesis.
- Netlist syntax shortcuts are synthesizable, but compilation context matters.

| SystemVerilog-2005/2009/2012                                                                                                                           |                                                                                                                                                         |                                                                                   |                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| assertions<br>test program blocks<br>clocking domains<br>process control                                                                               | mailboxes<br>semaphores<br>constrained random values<br>direct C function calls                                                                         | classes<br>inheritance<br>strings<br>references                                   | dynamic arrays<br>associative arrays<br>queues<br>checkers                                                               |
| interfaces<br>nested hierarchy<br>unrestricted ports<br>automatic port connect<br>enhanced literals<br>time values and units<br>specialized procedures | packed arrays<br>array assignments<br>unique/priority caself<br>void functions<br>function input defaults<br>function array args<br>parameterized types | break<br>continue<br>return<br>do-while<br>case inside<br>aliasing<br>const       | enum<br>typedef<br>structures<br>unions<br>2-state types<br>packages<br>\$unit<br>streaming<br>casting                   |
| ++ -- += -= *= /=<br>>>> <<< >>>> <<<<<br>&&    ~ & ~&<br>== ? != ?<br>inside<br>streaming<br>casting                                                  |                                                                                                                                                         |                                                                                   |                                                                                                                          |
| Verilog-2005                                                                                                                                           |                                                                                                                                                         |                                                                                   |                                                                                                                          |
| uwire                                                                                                                                                  | 'begin_keywords                                                                                                                                         | 'pragma                                                                           | \$clog2                                                                                                                  |
| Verilog-2001                                                                                                                                           |                                                                                                                                                         |                                                                                   |                                                                                                                          |
| ANSI C style ports<br>generate<br>localparam<br>constant functions                                                                                     | standard file I/O<br>\$value\$plusargs<br>'ifndef 'elsif 'line<br>@*                                                                                    | (* attributes *)<br>configurations<br>memory part selects<br>variable part select | multi dimensional arrays<br>signed types<br>automatic<br>"" (power operator)                                             |
| Verilog-1995 (created in 1984)                                                                                                                         |                                                                                                                                                         |                                                                                   |                                                                                                                          |
| modules<br>parameters<br>function/tasks<br>always @<br>assign                                                                                          | \$finish \$stopen \$fclose<br>\$display \$write<br>\$monitor<br>'define 'ifdef 'else<br>'include 'timescale                                             | initial<br>disable<br>events<br>wait # @<br>fork-join                             | wire reg<br>integer real<br>time<br>packed arrays<br>2D memory<br>begin-end<br>while<br>for forever<br>if-else<br>repeat |
|                                                                                                                                                        |                                                                                                                                                         |                                                                                   | begin-end<br>while<br>for forever<br>if-else<br>repeat                                                                   |

## Verilog discrete ports



## SystemVerilog interface ports



## The case() inside Decision Statement

- The **case()** inside statement replaces **casex** and **casez**.
- Bits set to X, Z or ? in the **case items** are “don't care” bits.
- Any X, Z or ? bits in the **case expression** are **not** don't cares.
- With **casez** and **casex**, X, Z of ? bits in the case expression are also considered don't cares – which is a serious problem.



```
case (opcode) inside
  8'b1??????? : ... // only compare most significant bit
  8'b????1111 : ... // compare lower 4 bits, ignore upper bits
  ...
default: Error("bad opcode");
endcase
```

If opcode has the value 8'b11111111, which branch should execute?

## What's the advantage?

- ✓ **case()** inside eliminates the serious **GOTCHA** of **casex** and **casez** than could lead to design bugs going undetected

## Page 2 — RTL coding rules, tool caveats, and what to memorize cold

| RTL feature                                                                  | Main benefit                                                                                     | Best-practice / gotcha                                                                                                                              |
|------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>always_comb</code> / <code>always_latch</code>                         | State combinational/latch intent explicitly; tools can verify the block matches that intent.     | <code>always_comb</code> runs at time 0 and includes function-internal reads in sensitivity; use instead of plain <code>always</code> in synth RTL. |
| <code>always_ff</code>                                                       | Documents flip-flop intent for sequential logic.                                                 | You still must write the event control correctly; tools can warn if block behavior is not FF-style.                                                 |
| <code>==?</code> / <code>!=?</code>                                          | Wildcard equality with masked bits in operand 2.                                                 | Synthesizes like a comparator with masked bits ignored; safer than loose <code>casex</code> habits.                                                 |
| <code>inside</code> / <code>case...inside</code>                             | Set-membership matching with ranges/lists and masked values.                                     | Only constant-expression sets are synthesizable. Authors recommend <code>case...inside</code> over <code>casez/casex</code> .                       |
| <code>unique</code> / <code>unique0</code> / <code>priority case</code>      | Puts <code>full_case</code> / <code>parallel_case</code> intent into the language with checking. | Still initialize outputs to avoid latches; these keywords catch many hazards, not every one.                                                        |
| <code>priority</code> / <code>unique</code> / <code>unique0</code> if...else | Brings the same intent checking to if/else ladders.                                              | Use when branches are intentionally priority or mutually exclusive.                                                                                 |
| <code>void function over task</code>                                         | Cleaner RTL helper style for side-effect free procedural helpers.                                | Task input defaults and ref args were not synthesizable in this tool era.                                                                           |
| <code>dot-name</code> / <code>dot-star</code>                                | Shorter instance hookups when names match.                                                       | Adds safety: explicit declarations and size matching. Dot-star needs both modules compiled together.                                                |
| <code>`begin_keywords</code>                                                 | Locks compilation to a language-version keyword set.                                             | Important when mixing older Verilog with SV keywords like <code>priority</code> .                                                                   |
| <code>timeunit</code> / <code>timeprecision</code>                           | Local replacement for file-scoped <code>`timescale</code> .                                      | Ignored by synthesis but strongly recommended for simulation hygiene in every module/interface.                                                     |
| <code>\$clog2</code> / <code>\$bits</code>                                   | Expression-size helpers for scalable widths.                                                     | Use them so declarations follow <code>params/types</code> instead of hard-coded constants.                                                          |
| <code>assertions</code>                                                      | Useful for RTL simulation checking/documenting intent.                                           | Immediate assertions were ignored/handled differently by these tools; do not rely on them as synthesized hardware.                                  |

### Historical tool snapshot from the 2013 material

| Construct                                                 | DC 2012.06 | Synplify 2012.09 |
|-----------------------------------------------------------|------------|------------------|
| <code>`begin_keywords</code> / <code>`end_keywords</code> | yes        | no               |
| package import before module port list                    | yes        | no               |
| <code>case...inside</code>                                | yes        | no*              |
| <code>unique/priority</code> on if...else                 | yes        | ignored          |
| parameterized task/function (classes)                     | yes        | no               |
| real data type                                            | no         | yes              |
| nets from typedef struct definitions                      | no         | yes              |
| immediate assertions                                      | ignored    | yes              |
| interface modport expressions                             | no         | yes              |

\*Paper note: Synplify-Pro added `case...inside` support later, after the version used here.

**Wish-list / not fully synthesizable then:** `uwire` synthesis support, `foreach`, task/function default inputs, ref args, `inside` with expressions, package chaining, extern module declarations, configurations, and generic/user-defined net types.

### The 15 recommendations — memorize these

|    |                                                                                                         |     |                                                                                                                                         |
|----|---------------------------------------------------------------------------------------------------------|-----|-----------------------------------------------------------------------------------------------------------------------------------------|
| 1. | Use logic for most ports/signals; forget wire/reg.                                                      | 9.  | Use <code>priority</code> / <code>unique0</code> / <code>unique</code> instead of <code>full_case</code> / <code>parallel_case</code> . |
| 2. | Use <code>uwire</code> for single-driver checking (language win, though synthesis support lagged here). | 10. | Use <code>priority</code> / <code>unique0</code> / <code>unique</code> with if...else when appropriate.                                 |
| 3. | Use <code>enum</code> for limited legal values.                                                         | 11. | Use <code>void function</code> instead of <code>task</code> in RTL.                                                                     |
| 4. | Use <code>struct</code> to collect related variables.                                                   | 12. | Use <code>dot-name</code> and <code>dot-star</code> shortcuts.                                                                          |
| 5. | Use <code>typedef</code> for consistent declarations.                                                   | 13. | Use interfaces for related bus signals.                                                                                                 |
| 6. | Use packages for shared declarations.                                                                   | 14. | Use <code>`begin_keywords</code> to lock the language version.                                                                          |
| 7. | Use <code>always_comb</code> / <code>always_latch</code> / <code>always_ff</code> .                     | 15. | Use local <code>timeunit/timeprecision</code> instead of <code>`timescale</code> .                                                      |
| 8. | Use <code>case...inside</code> instead of <code>casez/casex</code> .                                    |     |                                                                                                                                         |

**Fast memory order:** `logic` > `wire/reg`, `enum` > raw FSM params, `struct/typedef/package/interface` reduce mismatch, `always_*` > plain `always`, `inside` > `casez/casex`, `unique/priority` > old pragmas, `timeunit` > ``timescale`.

#### SystemVerilog adds many new synthesizable constructs:

- `==?` and `!=?` wildcard equality/inequality operators
- `inside` set membership operator
- `<<`, `>>` pack and unpack streaming operators
- `++` and `--` increment and decrement operators
- `+=`, `-=`, `*=`, `/=` ... assignment operators

```
if (data inside {0:255}) ... if data is between 0 to 255, inclusive
if (data inside {3'b101, 3'b111, 3'b1x1, or 3'b1z1}) ... if data is 3'b101, 3'b111, 3'b1x1, or 3'b1z1
a = { << ( b ) }; bit reverse – unpack bits of b and assign to a in reverse order
c = { << ( d ) }; byte reverse – unpack 8-bit chunks of d and assign in reverse order
```

#### What's the advantage?

- Model more RTL functionality in fewer lines of code



#### Fifteen Ways You Can Benefit from Using SystemVerilog in RTL designs

- Use `logic` for modules ports and most internal signals – forget `wire`, `reg`
- Use the `uwire` net type to check for and enforce single-driver logic
- Use `enumerated` types for variables with limited legal values
- Use `structures` to collect related variables together
- Use `user-defined` types to ensure consistent declarations in a design
- Use `packages` for declarations that are shared throughout a design
- Use `always_comb`, `always_latch` and `always_ff` procedural blocks
- Use `case...inside` instead of `casez` and `casex`
- Use `priority`, `unique0`, `unique` instead of `full_case`, `parallel_case`
- Use `priority`, `unique0`, `unique` with if...else when appropriate
- Use `void function` instead of task in RTL code
- Use `dot-name` and `dot-star` netlist shortcuts
- Use `interfaces` to group related bus signals
- Use ``begin_keywords` to specify the language version used
- Use a locally declared `timeunit` instead of ``timescale`



#### Type, Size and Sign Casting

- SystemVerilog adds casting operations to Verilog
  - `<type>(<expression>)` — cast expression to different data type
  - `<size>(<expression>)` — casts expression to a vector size
  - `signed(<expression>)` — casts expression to signed
  - `unsigned(<expression>)` — casts expression to unsigned

```
logic [31:0] a, y;
logic [ 5:0] b;
y = (a,a) >> b;

y = logic [31:0] ((a,a) >> b);
```

Rotate a by b number of times

Will get warning from lint checkers and synthesis because LHS is 32 bits and RHS is 64 bits

cast the operation result to 32 bits so that the RHS and the LHS are the same size

#### What's the advantage?

- Documents intent that a change in type, size or sign is intended
- Can eliminate size and type mismatch warnings



#### Module Instance Port Connection Shortcuts

- Verilog netlist port connections must name both the port and the net connected to it

```
module dff (output q, qb;
input clk, rst, pre);
dff dff1 (.clk(clk), .rst(rst), .pre(pre), .d(d[0]), .q(q[0]));
```
- SystemVerilog adds `.name` and `.shortcuts`
  - `.name` connects a port to a net of the same name

```
dff dff1 (.clk, .rst, .pre, .d(d[0]), .q(q[0]));
```
  - `.shortcuts` automatically connects all ports and nets with the same name

```
dff dff1 (.*, .q(q[0]), .d(d[0]), .qb(qb));
```

#### What's the advantage?

- Reduce typing (and typos) when connecting design blocks
- Built-in checking prevents connection mismatches